Data Analysis with Python

COURSE EXERCISES

Exercise 1 : NumPy 2

Exercise 2 : Pandas Intro 5

Exercise 3 : Selecting and Filtering 7

Exercise 4 : Cleaning and Transforming 10

Exercise 5 : Advanced Indexing 14

Exercise 6 : Rearranging Data 17

Exercise 7 : Combining Data 20

Exercise 8 : Grouping Data 24

Exercise 9 : Visualization 27

Exercise 10 : Time Series 30

Exercise 11 : Integration Case 34

Exercise 13 : Data Formats and Tools 38

Exercise 1 : NumPy

Open `exercises/01-numpy/exercise.ipynb` in JupyterLab. The solution is in `solutions/01-numpy/solution.ipynb`.

Run the first cell : it imports NumPy as `np`. The values that a step gives you are already assigned in the notebook.

Step 1 : Arrays

Print the NumPy version. Convert the list `numbers = [12.23, 13.32, 100, 36.32]` into a one-dimensional array and print its `ndim`.

Step 2 : Creating arrays

Create a 4x2 matrix with the values 5 to 12 with `np.arange` and `reshape`. Create a vector of eight zeros and set its third value, at position 2, to 4.

Step 3 : Broadcasting

Reshape the integers 1 to 6 into `readings` with shape `(3, 2)`. The array `scale` has shape `(3, 1)` : `np.array([[1], [10], [100]])`. Predict the shape of `readings * scale`, then run it. Which axis of `readings` does `scale` match?

Try `readings + np.array([1, 2, 3])` inside `try` / `except ValueError` and print the error. Why are the shapes `(3, 2)` and `(3,)` incompatible?

Compute `readings.sum(axis=0)` and `readings.sum(axis=1)`. Which of the two has one value per row?

Step 4 : Views and copies

`original = np.arange(5, 30)` holds the integers 5 to 29. Print it in reverse order, so that the first element becomes the last.

Take `view = original[2:5]` and an independent copy of the same slice with `.copy()`. Set `view[0] = -1` and print the first five values of `original` and the copy. Check with `np.shares_memory()` which of the two shares memory with `original`. When do you need a copy before you change a slice?

Step 5 : Data types

Convert `whole_numbers = [3, 6, 9, 12]` to an array with `dtype=np.float64`, divide it by 4 and print the result and its dtype. Convert the list `as_list` and the tuple `as_tuple` to arrays with `np.asarray`.

For `measured = np.array([2, 4, 6, 8], dtype=np.int32)`, print the number of elements, the size of one element in bytes and the total number of bytes.

Step 6 : Appending and filling

Append the values 35 to 75, in steps of 10, to `collected = np.array([5, 15, 25])` with `np.append`. Print both arrays. Does `np.append` change `collected`?

Create an empty 2x5 array with `np.empty` and a 4x2 array full of 0.5 with `np.full`. What values does the empty array contain?

Step 7 : Unique values

`repeated` holds six country codes. Print its unique elements and how often each occurs.

Step 8 : Positions and masks

For `values = np.array([[4, 17, 9], [12, 3, 21]])`, find the position of the minimum and of the maximum in each row with `argmin`, `argmax` and `axis=1`.

Find the elements greater than 10 : print their positions with `np.nonzero` and their values with a Boolean mask.

Step 9 : Spacing and flattening

Create a one-dimensional array of 13 evenly spaced values from 1 to 4, both included. Flatten `nested = np.array([[1, 2], [3, 4], [5, 6]])` into a one-dimensional copy.

Step 10 : Z-scores

`scores = np.array([[70, 80], [90, 60], [80, 70]])` holds two test scores, in the columns, for three students, in the rows. Standardize each column : subtract the column mean and divide by the column standard deviation, both with `axis=0`. Which student, by row position, lies furthest above the average in the first test? Use `argmax`.

Exercise 2 : Pandas Intro

Open `exercises/02-intro-pandas/exercise.ipynb` in JupyterLab. The solution is in `solutions/02-intro-pandas/solution.ipynb`.

Run the first cell : it imports pandas and sets `DATA` to the folder with the exercise files.

Step 1 : Inspection

Read `world_ind_pop_data.csv`, World Bank population data from 1960 to 2014, into a DataFrame `df`. Look at the first and last rows with `df.head()` and `df.tail()`. Which years and countries does the file start and end with?

Step 2 : Missing values

Show the data type and the number of non-missing entries of each column with `df.info()`. Count the missing values per column with `df.isna().sum()`. Does any column have missing values? Where does `info()` show the same answer?

Step 3 : Broadcasting

Build a DataFrame of Dutch cities, with the city name in one column and the country code in the other :

- Make a list `cities` with the names of five Dutch cities
- Make a string `country` with the value `'NL'`
- Make a dictionary with the pairs `'country': country` and `'city': cities`
- Build a DataFrame `df` from the dictionary and print it

How often does `'NL'` appear in `df`, and where do the extra copies come from?

Step 4 : Column labels

Read `world_population.csv` into `df1` with `pd.read_csv()`. Read the same file again into `df2`, this time with `header=0` and `names=new_labels`, where `new_labels = ['year', 'population']`. Print both DataFrames and compare the column names. What does `header=0` tell pandas about the first line of the file?

Step 5 : A messy file

`messy_stock_data.tsv` has a misleading extension : its fields are separated by spaces. It also starts with three introductory lines and contains comment lines that start with `#`. Open the file in JupyterLab's file browser before you write any code.

Read the file, whose path is in `file_messy`, into `df1` with the default options of `pd.read_csv()` and print `df1.head()`. Read it again into `df2` with `delimiter=' '`, `header=3` and `comment='#'`. The clean table has four rows, IBM, MSFT, GOOGLE and APPLE, and 13 columns : `name` and the months January to December.

Write `df2` to the tab-separated file `file_clean.tsv` with `sep='\t'` and `index=False`. Read it back with `sep='\t'` and check that the columns and values are unchanged. Then write `df2` to `file_clean.xlsx` with `to_excel(..., index=False)`.

Exercise 3 : Selecting and Filtering

Open `exercises/03-selecting-and-filtering/exercise.ipynb` in JupyterLab. The solution is in `solutions/03-selecting-and-filtering/solution.ipynb`.

Run the first cell : it reads `pennsylvania2012_turnout.csv`, the results of the 2012 US presidential election in the 67 counties of Pennsylvania, into `election`, with the county names as row labels. The columns are `state`, `total` (votes cast), `Obama` and `Romney` (vote shares in percent), `winner`, `voters` (registered voters), `turnout` (percent) and `margin` (winning margin in percent).

Step 1 : One value

Look at `election.head()`. Select the winner in Bedford county with `.loc` and print it.

Step 2 : Positions

Find the row position `x` of Bedford and the column position `y` of `winner`, so that `election.iloc[x, y]` returns the same value as `election.loc['Bedford', 'winner']`. Count the positions in `election.head()` ; the first position is 0. Print both selections and compare them with `==`.

Step 3 : Columns

Create a DataFrame `results` with the columns `winner`, `total` and `voters`, in that order, and print `results.head()`.

Step 4 : Row slices

The counties are in alphabetical order. Select the counties from `'Perry'` to `'Potter'` as `p_counties`. Select the same counties in reverse order, from `'Potter'` to `'Perry'` with a step of `-1`, as `p_counties_rev`. Print both.

Step 5 : Column slices

Select three column ranges with `.loc` and print the first five rows of each :

- From the first column up to `'Obama'`, as `left_columns`
- From `'Obama'` to `'winner'`, as `middle_columns`
- From `'Romney'` to the last column, as `right_columns`

Step 6 : Lists

Select the rows in `rows = ['Philadelphia', 'Centre', 'Fulton']` and the columns in `cols = ['winner', 'Obama', 'Romney']` with `.loc`. Assign the result to `three_counties` and print it.

Step 7 : Filters

Create a Boolean Series `high_turnout` that is `True` where `turnout` is greater than 70. Filter `election` with it into `high_turnout_df` and print it. How many counties have a turnout above 70?

Step 8 : Flag column

A margin below 1 % is a close result. Create a Boolean Series `too_close` for these counties and store it in `election` as a new column `too_close`. Print the `winner` and `margin` of the flagged counties : you find Berks, Centre and

Chester.

Combine two conditions : which counties have a turnout above 70 and a margin below 5?

Step 9 : Summaries

Answer each question with one expression :

- How many counties did each candidate win? Use `value_counts()` on `winner`
- Which three counties had the highest turnout? Use `nlargest`
- Which county had the smallest margin? Use `idxmin()` on `margin`
- How many counties had a turnout between 60 and 70 percent? Use `between` and `sum()`
- What is the median turnout, and how does it compare with the mean?
- What is the turnout in `Philadelphia`, `Allegheny` and `Centre`? Select them with `isin` on the index

If time permits :

- Which counties did Romney win with a turnout above the median county turnout? Sort them from the highest to the lowest turnout and count them

Exercise 4 : Cleaning and Transforming

Open `exercises/04-cleaning-and-transforming/exercise.ipynb` in JupyterLab. The solution is in `solutions/04-cleaning-and-transforming/solution.ipynb`.

Run the first cell : it imports pandas and sets `DATA` to the folder with the exercise files. The steps use three of them :

- `titanic.csv` : one row per passenger of the Titanic, 1309 rows
- `pittsburgh2013.csv` : daily weather in Pittsburgh in 2013, from Weather Underground
- `pennsylvania2012_turnout.csv` : the election results from Exercise 3

Step 1 : Dropping

Read `titanic.csv` into `titanic`. Select the `age` and `cabin` columns into a DataFrame `df` and print its shape. Drop the incomplete rows of `df` with `dropna(how='any')` and print the shape, then do the same with `how='all'`. Why do the two shapes differ?

Count the missing values in each column of `titanic` with `isna().sum()` and assign the result to `missing_per_column`. Keep the columns with at most 1000 missing values as `titanic_reduced`, with `dropna(axis='columns', thresh=...)`. `thresh` is the minimum number of non-missing values, here `len(titanic) - 1000`. Print the remaining column names. Which columns were dropped?

Step 2 : Filling

Count the passengers without a port of embarkation in `embarked` with `isna().sum()`. Fill the gaps with the text `'unknown'` and store the result back in `embarked`. Print `titanic['embarked'].value_counts()`. Why is `'unknown'` a better fill than the most common port?

Step 3 : Apply

Read `pittsburgh2013.csv` into `weather`. Write a function `to_celsius(f)` that converts degrees Fahrenheit to Celsius with `(f - 32) * 5 / 9`.

Call `.apply(to_celsius)` on the columns `['Mean TemperatureF', 'Mean Dew PointF']` and assign the result to `df_celsius`. Rename its columns to `['Mean TemperatureC', 'Mean Dew PointC']` and print `df_celsius.head()`.

`apply` calls `to_celsius` once per column. Call `to_celsius` directly on the same two columns and compare the values. Why are they the same?

Step 4 : Map

Read `pennsylvania2012_turnout.csv` into `election` with `index_col='county'`. The dictionary `red_vs_blue` gives each candidate a color. Map the `winner` column with it into a new column `color` and print `election.head()`.

Map `sample_winners` with the same dictionary and print the result. Which values become `NaN`? Why is that safer than a rule that returns `'red'` for every name other than `'Obama'`?

Step 5 : Z-scores

A z-score is the number of standard deviations that a value lies above the mean. Add a column `turnout_zscore` with the z-score of `turnout`, computed with arithmetic on whole columns and without `apply`. Print the five counties with the highest z-score. Does a high z-score prove that a turnout figure is wrong?

Step 6 : Text

One column name of `weather` starts with a space : print `weather.columns.tolist()` to find it. Strip the spaces from all column names with `weather.columns.str.strip()`.

`Events` describes the weather of each day, such as `'Rain'` or `'Fog-Snow'`, and is missing for a quiet day. Count the days with snow with `str.contains('Snow')` and `sum()`. Fill the missing events with `'None'` and print the `value_counts()` of the column.

Step 7 : Types

Create a table `daily` with the `Date`, `Max TemperatureF`, `Min TemperatureF` and `Events` columns of `weather`. Rename the temperature columns to `max_f` and `min_f`.

Convert `Date` from text to dates with `pd.to_datetime()` and check the result with `daily.dtypes`. Add a column `range_f`, the difference between `max_f` and `min_f`, and print the three days with the largest range with `nlargest`.

If time permits :

- On how many days with snow did the maximum temperature rise above freezing, 32 °F? What share of all days with snow is that?

Exercise 5 : Advanced Indexing

Open `exercises/05-advanced-indexing/exercise.ipynb` in JupyterLab. The solution is in `solutions/05-advanced-indexing/solution.ipynb`.

Run the first cell : it imports pandas and sets `DATA` to the folder with the exercise files. The steps use two small files with made-up figures for a bike rental company :

- `rentals.csv` : rentals per month from January to June for three bike types, `city_bikes`, `e_bikes` and `cargo_bikes`, with a `month` column. One cargo-bike count is missing
- `rentalspercity.csv` : the January and February rentals split over three cities, with a `city` column : Amsterdam (`AMS`), Rotterdam (`RTM`) and Utrecht (`UTR`)

Step 1 : Labels and positions

Read `rentals.csv` into `rentals` with `index_col='month'`. Select the February city bikes with `.loc['Feb', 'city_bikes']` and with `.iloc[1, 0]` ; both return 287. Which accessor uses a label, and which a position?

Run the three statements below, each in its own cell. Wrap the first two in `try` / `except TypeError` and print the error :

```
rentals.index[0] = 'JAN'
rentals.index[0] = rentals.index[0].upper()
rentals.index = range(len(rentals))
```

Which statement succeeds, and what happens to the month labels?

Step 2 : Alignment

Read `rentals.csv` again with its month index. Uppercase all labels with `rentals.index.str.upper()`, assign the result to `rentals.index` and print `rentals`.

Late rental reports arrive as `corrections`, a Series in a different month order. Add `corrections` to `rentals['city_bikes']` as `aligned`. In January-to-June order the values are 322, 290, 412, 517, 622 and 680.

Drop the May correction with `corrections.drop('MAY')`, repeat the addition as `partial` and find the month without a value. Why does the order of the corrections not matter, while a missing label does?

Step 3 : Index names

Read the table again and uppercase its month labels. Set `rentals.index.name = 'MONTH'` and `rentals.columns.name = 'BIKE_TYPE'` and print the table. What do these names describe, compared with labels such as `JAN` and `city_bikes`?

Step 4 : Index from a column

Read `rentals.csv` without `index_col` into `raw`. Build `rentals` with `set_index('month')` and check `rentals.index.is_unique`. March has 405 city-bike rentals.

Reverse the rows with `raw.iloc[::-1]` and build `reordered` in the same way. Print `reordered.loc['Mar', 'city_bikes']` and `reordered.iloc[0]['city_bikes']`. Which of the two changed? Why would a fixed list of month names, assigned to the reversed rows, mislabel them?

Turn the index of `reordered` back into a column with `reset_index()` and print the result.

Step 5 : Duplicate labels

Read `rentalspercity.csv` into `by_city` with `city` as the index and check `by_city.index.is_unique`.

Select `by_city.loc['RTM']` as `rotterdam`. How many rows does it return? Compare it with `by_city.iloc[2]`, which selects one row by position.

Try `by_city.reindex(['AMS', 'RTM', 'UTR'])` inside `try` / `except ValueError` and print the error. Why can't pandas pick one row per city?

If time permits :

Two items for the optional MultiIndex slides :

1. Read `rentalspercity.csv` into `by_city` with `['city', 'month']` as the index, check that the index is unique and sort it with `sort_index()`. Select `by_city.loc[['AMS', 'UTR']]` as `selected` and `by_city.loc['AMS':'UTR']` as `ranged`. How many rows does each have? In which order do the months sort?
2. On the same index, select :
   - The Rotterdam January row as `rtm_jan` ; its city-bike value is 97
   - The Amsterdam and Utrecht February rows as `ams_utr_feb`, with a list inside the tuple
   - Every city's February row as `all_feb`, with `(slice(None), 'Feb')` and again with `by_city.xs('Feb', level='month')`

   Print the February city-bike total. Does it match the February value in `rentals.csv` ?

Exercise 6 : Rearranging Data

Open `exercises/06-rearranging-data/exercise.ipynb` in JupyterLab. The solution is in `solutions/06-rearranging-data/solution.ipynb`.

Run the first cell : it reads `users.csv` into `users`, the visitors and newsletter signups of a band's blog per weekday and city. `index_col=0` discards the exported row numbers :

```
  weekday    city  visitors  signups
0     Sun   Paris       139        7
1     Sun  Berlin       237       12
2     Mon   Paris       326        3
3     Mon  Berlin       456        5
```

Step 1 : Pivot

Decide which variable labels the rows (`weekday`), which labels the columns (`city`) and which fills the cells (`visitors`), and sketch the result on paper.

Pivot `users` with `weekday` as the index, `city` as the columns and `visitors` as the values, and print the result. Does it match your sketch?

Step 2 : Pivoting all columns

Pivot `users` again with `signups` as the values and print it.

Pivot once more without `values`. Print the result : both `visitors` and `signups` are pivoted, under two-level column labels. Select the `signups` table from it.

Step 3 : Unstack and stack

Set the index of `users` to `['city', 'weekday']`. Unstack the `weekday` level into `byweekday` and print it. Stack `byweekday` by `weekday` and compare the result with the indexed `users`.

Remove the Berlin Sunday row with `drop(index=('Berlin', 'Sun'))` and call the result `sparse`. Unstack and stack the weekday level of `sparse`. How many rows does the result have, and what is in the extra row? Why is that row different from zero visitors?

Step 4 : The other level

Unstack the `city` level of the indexed `users` into `bycity` and print it. Stack it again by `city`. Are the rows in the same order as in `users`?

Step 5 : Index order

Stack `bycity` back into `newusers`. Swap the two index levels with `.swaplevel(0, 1)` and print the result. Is the index sorted? Sort it with `sort_index()` and compare it with `users.sort_index()` using `.equals()`.

Step 6 : Melt

Take the visitors table from Step 1 and turn its weekday index back into a column with `reset_index()`. Melt it with `id_vars='weekday'`, `var_name='city'` and `value_name='visitors'` into one row per weekday and city. How many rows do you get?

Step 7 : Pivot tables

Read `titanic.csv`. Each passenger has a `pclass` (1, 2 or 3), a `sex` and `survived` (1 or 0).

Compute the survival rate per class and sex with `pivot_table`, with `pclass` as the index, `sex` as the columns, `survived` as the values and `aggfunc='mean'`. Repeat it with `aggfunc='count'` to see how many passengers each rate rests on. Why does `pivot` fail on this data?

If time permits :

- Read `users.csv` again as in the first cell. Reshape it so that each row is a city and each column a weekday, with the signups as values, and add a column `total` with each city's signups

Exercise 7 : Combining Data

Open `exercises/07-combining-data/exercise.ipynb` in JupyterLab. The solution is in `solutions/07-combining-data/solution.ipynb`.

Compute the 2024 population of each World Bank income group and its share aged 65 and over. Count each economy once, and keep every row that you leave out visible in an audit.

Run the first cell : it sets `DATA` to the course's `datasets` folder and `SOURCES` to the two indicator files with their labels. Read `datasets/wb-population-README.md` first. The files are unchanged World Bank downloads, retrieved on 10 October 2026 :

- `wb-wdi-population-total.csv` and `wb-wdi-population-65plus.csv` : World Development Indicators, with one row per economy or aggregate and one column per year
- `wb-class-economies.csv` : the income classification, one row per economy or aggregate
- `wb-class-groups.csv` : the group memberships, one row per economy and group

Step 1 : Appending

Read the two indicator files into the list `frames`. Open one of them in a text editor first : four metadata lines precede the header, so use `skiprows=4`. Every line ends with a comma, which creates an empty column `Unnamed: 70`. Check that it is empty and drop it.

Add the labels of `SOURCES` as a `source` column and concatenate the frames once into `indicators`, with a fresh integer index. Check that :

- `value_counts()` on `source` gives 266 rows per file, 532 in total
- No pair of `Country Code` and `Indicator Code` occurs twice

- Every country code is three capital letters : `str.fullmatch(r'[A-Z]{3}')`

Count the missing 2024 values as `missing_2024`. Which row is empty? Why is an empty value different from zero people?

Step 2 : Duplicate keys

Keep the `Country Code`, `Indicator Code` and `2024` columns of `indicators` as `population_2024`. Read `wb-class-groups.csv` into `groups`. Select every row whose `WB_Country_Code` occurs more than once, keeping all occurrences, as `duplicate_groups`, and print the memberships of the Netherlands.

Try a left merge of `population_2024` with `groups` on `Country Code` and `WB_Country_Code`, with `validate='many_to_one'`. Catch only `pd.errors.MergeError` and print its message. Why does a membership list repeat every economy? Which of the two tables must have unique keys for this merge?

Step 3 : Validated merge

Read `wb-class-economies.csv` into `economies`. Find and drop its one completely empty row with `isna().all(axis=1)`, and check that `Code` is complete and unique.

Merge `population_2024` with `economies` into `combined`, with `how='left'`, `validate='many_to_one'` and `indicator=True`. Check that `combined` has as many rows as `population_2024` and the same 2024 total. Each country appears twice on the left, once per indicator : why does the validation accept that?

Step 4 : Audit

Split `combined` into three parts and check that together they hold all its rows :

- `unmatched` : the `left_only` rows, whose code is not in `economies`
- `aggregates` : the matched rows without an income group, such as regions and the world
- `economies_2024` : the rows with an income group

Repeat the merge as `outer_audit`, with `how='outer'` and the same validation. Select its `right_only` rows as `unused_economies` : economies without a row in the indicator files.

Compute three totals for 2024 :

- `world` : the total population of `WLD`
- `economy_total` : the total population summed over `economies_2024`
- `gap` : the difference between the two

Which economy explains the gap? Compare the High income aggregate, `HIC`, with the sum of its member economies, then look at `unused_economies`. Explain in the answer cell whether leaving out the aggregates and the unmatched rows loses population.

Step 5 : Income groups

Put the two indicators side by side per income group with `pivot_table(..., aggfunc='sum')` on `economies_2024`, as `by_income`, and add the column `share_65plus`. Why is a sum safe here, while the same call on a table that repeats an economy would hide a defect? The slide *Aggregating Duplicate Pairs* shows the risk.

If time permits :

1. Melt the year columns of the total-population file into `long`, keeping `Country Code` and `Country Name`, with the columns `year` and `population`. Predict the number of rows, then check it. Look up the Netherlands in 2024 by its labels : 17,993,485. Pivot `long` back into `restored` and look up the Netherlands in 2024 and in 1960 : 11,486,631.
2. The notebook builds `bad`, an unchecked left join of the 2024 population of the economies onto `groups`. Compute `original_total`, `bad_total` and `extra`. Count the rows per economy with `value_counts()` on `Country Code` as `repeats`, and find the smallest and largest count. Explain in two or three sentences in the answer cell which table repeats its keys, how often, why the total grows and what repairs it. Does `validate='many_to_many'` show that the result is correct?

Exercise 8 : Grouping Data

Open `exercises/08-grouping-data/exercise.ipynb` in JupyterLab. The solution is in `solutions/08-grouping-data/solution.ipynb`.

Run the first cell : it reads `titanic.csv` into `titanic`, one row per passenger. The steps use these columns :

- `pclass` : ticket class 1, 2 or 3
- `survived` : 1 or 0
- `sex`, `age`, `fare` and `cabin`
- `embarked` : the port where the passenger boarded, `S` Southampton, `C` Cherbourg or `Q` Queenstown

Step 1 : Counting

Group `titanic` by `pclass` into `by_class` and count the non-missing `survived` values per class as `count_by_class`. Group by `embarked` and `pclass` together and count `survived` again as `count_mult`. Print both.

Step 2 : size and count

`count()` counts non-missing values ; `size()` counts rows. Print `by_class.size()` and `by_class['age'].count()`. How many passengers have no recorded age?

`count_mult` adds up to 1307 of the 1309 passengers. Group by `['embarked', 'pclass']` again with `dropna=False`, compute `size()` as `size_mult` and check that `size_mult.sum()` equals `len(titanic)`. Which passengers were missing from `count_mult` ?

Step 3 : Empty categories

Convert `embarked` to a categorical with the categories `port_order`, the order in which the Titanic called at the ports. Select the passengers aged 60 or over as `seniors`.

Group `seniors` by `['embarked', 'pclass']` and count `survived`, once with `observed=True` and once with `observed=False`. Which combinations appear only with `observed=False`? How does their 0 differ from a measured zero?

Step 4 : Several aggregates

Group by `pclass`, select `age` and `fare`, and aggregate with `['max', 'median']`. Print the oldest age per class with `.loc[:, ('age', 'max')]` and the median fare per class with `.loc[:, ('fare', 'median')]`.

Compute the same two results with named aggregation, `oldest_age=('age', 'max')` and `median_fare=('fare', 'median')`, and print them.

Step 5 : Filling by group

Fill each missing age with the median age of the passengers of the same sex and class. Work on a copy, `filled = titanic.copy()`, so that `titanic` keeps its missing ages for Step 7 :

- Save a copy of `filled['age']` as `original_age`
- Add a column `age_imputed` that is `True` where the age is missing
- Write a function `impute_median(series)` that returns `series.fillna(series.median())`
- Group `filled` by `['sex', 'pclass']` and assign the result of `transform(impute_median)` on `age` back to `filled['age']`

Print the number of filled ages. Check that every known age is unchanged : compare `filled.loc[~filled['age_imputed'], 'age']` with `original_age.dropna()` using `.equals()`. Why keep the `age_imputed` column for later analyses?

Step 6 : Apply

The notebook defines `c_deck_survival(group)`, the survival rate of the passengers in a group whose cabin is on deck C. Group by `sex`, select the `cabin` and `survived` columns and call `.apply(c_deck_survival)`.

Step 7 : Derived groups

Which share of the children under 10 survived in each class?

Create the Boolean Series `titanic['age'] < 10` and map it to labels with `.map({True: 'under 10', False: '10 or older'})`. A missing age compares as `False` and would land in `'10 or older'` : relabel those passengers with `.where(titanic['age'].notna(), 'age unknown')`. Name the result `under10` and print its `value_counts()`.

Group by `under10` and aggregate `survived` with `['mean', 'size']`. Group by `under10` and `pclass` together. Do the passenger counts add up to 1309?

Step 8 : Filtering groups

The first letter of `cabin` is the deck : `titanic['cabin'].str[0]`. Store it as a column `deck`.

Keep the decks with at least 20 passengers with `groupby('deck').filter(...)`. Compute the survival rate and the number of passengers per remaining deck with `agg(['mean', 'size'])`, and sort the decks from the highest to the lowest survival rate. How many passengers have no deck?

Exercise 9 : Visualization

Open `exercises/09-matplotlib/exercise.ipynb` in JupyterLab. The solution is in `solutions/09-matplotlib/solution.ipynb` .

Run the first cell : it reads the World Bank file `world_ind_pop_data.csv` from Exercise 2 and pivots it into `urban` , with one row per `Year` from 1960 to 2014 and one column per country : `Brazil` , `China` , `India` and `Nigeria` . The values are the urban population as a percentage of each country's population. Steps 7 and 8 use `tips.csv` .

Create every figure with `fig, ax = plt.subplots()` or `fig, axes = plt.subplots(rows, columns)` and draw with `ax.` methods.

Step 1 : Line chart

Create `fig, ax = plt.subplots()` . Plot `urban['China']` against `urban.index` in blue and `urban['India']` in red, both on `ax` .

Why does a line chart suit these yearly percentages? What can it not tell you about the number of people who live in cities?

Step 2 : Side by side

Create `fig, axes = plt.subplots(1, 2)` . Plot China in blue on `axes[0]` and India in red on `axes[1]` . Give each panel a title with `set_title` and call `fig.tight_layout()` so that the titles and labels fit.

Compare the y-axes of the two panels. Why do the two lines look alike?

Step 3 : Grid and style

Print `plt.style.available`. Inside `with plt.style.context('bmh'):`, create `fig, axes = plt.subplots(2, 2)`. Plot Brazil in green on `axes[0, 0]`, China in blue on `axes[0, 1]`, India in red on `axes[1, 0]` and Nigeria in purple on `axes[1, 1]`. Give each panel a title and call `fig.tight_layout()`.

Step 4 : Limits and saving

Plot China and India on one `ax`. Give each line a `label`, draw one solid and one dashed, and call `ax.legend()`. Set the x-range to 1980–2014 with `ax.set_xlim()` and the y-range to 0–60 with `ax.set_ylim()`, and label both axes. Save the figure as `xlim_and_ylim.png` with `fig.savefig()`. Would the chart still work when printed in grayscale?

Make the same chart again, but set both ranges with one call, `ax.axis((1980, 2014, 0, 60))`, and save it as `axis_limits.png`.

Step 5 : Legend and annotation

Plot all four countries with their names as labels. Place the legend in the upper left with `ax.legend(loc='upper left')` and label the y-axis `'Urban population (% of total)'`. Does the legend cover a line? Try another `loc` if it does.

Find the first year in which more than half of China's population lived in cities, as `crossed`, and China's share in that year, as `share`. A Boolean mask on `urban['China']` selects the years from `urban.index`. In a new figure, plot the China line and mark that point with `ax.annotate('Over 50% urban', ...)`, with `xy=(crossed, share)`, `xytext=(crossed - 25, share)` and `arrowprops=dict(facecolor='black')`.

Step 6 : pandas plotting

Plot all four countries of `urban` with one call to `.plot(ax=ax)`. Add axis labels and a title that states what the chart shows.

Make a second figure with `plot(subplots=True, layout=(2, 2))`, one panel per country. Which of the two figures compares the countries better, and which shows each country's changes better?

Step 7 : Seaborn

Read `tips.csv`, with the columns `total_bill`, `tip`, `sex`, `day` and `time`. Draw the mean tip per day with `sns.catplot`, with `kind='bar'`, `x='day'`, `y='tip'`, `hue='sex'`, `col='time'`, `errorbar=('ci', 95)` and `seed=0`. Save the grid with `g.savefig('tips_by_time.png')`.

A bar does not show how many bills its mean rests on. Count the bills per `time`, `day` and `sex` with `groupby(..., observed=True).size().reset_index(name='n')` and write the table to `tips_group_sizes.csv` without the index. Do the counts add up to all 244 bills? Which bar rests on a single bill?

Step 8 : Chart choice

Is a tip proportional to the bill? Answer with a chart of `tips` from this module, with a title that states what you found, and save it as `tip_vs_bill.png`. Explain in two sentences in the answer cell why you chose this chart and what it cannot show.

Exercise 10 : Time Series

Open `exercises/10-time-series/exercise.ipynb` in JupyterLab. The solution is in `solutions/10-time-series/solution.ipynb`.

Run the first cell : it imports pandas and sets `DATA` to the `datasets` folder. The steps use two files :

- `weather_data_austin_2010.csv` : hourly temperature, dew point and pressure readings for Austin in 2010. The file states neither its source nor its time zone
- `austin_airport_departure_data_2015_july.csv` : the scheduled Southwest Airlines departures from Austin-Bergstrom airport in July 2015, from the Bureau of Transportation Statistics, in local time

Assume that the departure file lists every scheduled Southwest departure from Austin in July 2015.

Step 1 : Parsing

Read `weather_data_austin_2010.csv` into `weather`. Convert the `Date` column with `pd.to_datetime()` and `format='%Y%m%d %H:%M'` and make it the index with `set_index()`. Print `weather.head()` and the dtype of the index.

Assign `weather.index.is_monotonic_increasing` to `is_sorted` and the number of duplicated timestamps to `n_duplicates`, and print both. Why does the order matter before you slice by date?

Step 2 : Gaps

A year has 8760 hours. Create the complete hourly range from the first to the last timestamp of `weather` with `pd.date_range()` and `freq='h'` as `expected`, and print the lengths of `expected` and `weather`.

Select the timestamps of `expected` without a reading with `expected.difference()` as `missing` and print them. On which date does the gap fall, and what happens to the clocks in Texas on that date?

Step 3 : Filling

Assign `weather['Temperature'].asfreq('h')` to `hourly` and its number of missing values to `n_filled`.

Fill the gap twice : with `.ffill()` as `filled_ffill`, and with `.interpolate(method='time')` as `filled_interp`. Print the recorded, forward-filled and interpolated values from 2010-03-14 01:00 to 05:00 side by side. Which estimate suits a temperature better? Would you fill a gap of a week in the same way?

Step 4 : Downsampling

Resample `weather['Temperature']` to days and aggregate with `.agg(['min', 'max', 'mean', 'count'])` as `daily`. Select the days with fewer than 24 readings as `incomplete_days` and print them.

Resample the temperatures to month ends with the alias `'ME'` and `.mean()` as `monthly_mean`, and print it rounded to one decimal. Which month is the warmest? pandas 3 no longer accepts the old alias `'M'`.

Step 5 : Rolling mean

Select the August 2010 temperatures with `weather.loc['2010-08', 'Temperature']` as `august`, and resample them to daily maxima as `daily_highs`.

Smooth the daily highs with `daily_highs.rolling(window=7).mean()` as `smoothed` and print both Series side by side. Why are the first six values of `smoothed` missing?

Step 6 : Event counts

Read `austin_airport_departure_data_2015_july.csv` into `departures` with `skiprows=15`. Strip the spaces around the column names with `.str.strip()` and drop the footer row, which has no `'Flight Number'`.

Join `'Date (MM/DD/YYYY)'`, a space and `'Scheduled Departure Time'` into one string per row. Parse the strings with `format='%m/%d/%Y %H:%M'` into the Series `scheduled` and make it the sorted index of `departures`.

Four flights have an `'Actual Departure Time'` and a `'Wheels-off Time'` of `'00:00'` : they never departed, so their delay of 0 was never measured. Mark them in the Boolean Series `not_departed`. Assign the `'Departure Delay(Minutes)'` column to `delay`, with these four rows set to NaN by `.where()`.

Count the scheduled departures per day with `departures.resample('D').size()` as `daily_departures` and print its summary statistics.

Step 7 : Hourly counts

Build the DataFrame `july4` for 4 July 2015 with 24 rows, 00:00 to 23:00, and two columns : `'departures'`, the number of scheduled departures, and `'mean_delay'`, the mean measured delay per scheduled hour. Complete the outline in the notebook : reindex onto `pd.date_range('2015-07-04', periods=24, freq='h')` and fill only the departure counts with 0. Print `july4`. How many hours have no scheduled departure? Why would a mean delay of 0 be wrong for those hours?

Step 8 : Time zones

Sort `scheduled` from Step 6 with `.sort_values()`. Localize it to `'America/Chicago'` as `scheduled_local`, convert that to `'UTC'` as `scheduled_utc`, and print the first rows of both.

If time permits :

Try `weather.index.tz_localize('America/Chicago')` inside `try` / `except ValueError` and print the error. Localize again with `nonexistent='NaT'` and `ambiguous='NaT'` as `weather_local`, and select the timestamps of `weather` that became NaT as `unplaced`. Compare them with the gap from Step 2. Explain in the answer cell which clock the file seems to use.

Without a solution :

- Plot `daily['min']` and `daily['max']` on one Axes, with axis labels, units and a legend, and save the figure with `fig.savefig()`
- Resample `delay` to weeks ending on Sunday with `'W'` and print the mean delay beside the number of measured departures per week
- Write two sentences about the departure delays on 4 July, with the number of flights behind each figure

Exercise 11 : Integration Case

Open `exercises/11-integration-case/exercise.ipynb` in JupyterLab. The solution is in `solutions/11-integration-case/solution.ipynb`.

Answer the question of the slides : how many road deaths per million inhabitants did 33 European countries record in 2024, per age group? Steps 1–12 follow the twelve steps on the slides ; try each step yourself before you look back. Step 13 audits a flawed analysis.

Read `datasets/eurostat-road-safety-README.md` first. The files are unchanged Eurostat extracts, retrieved on 10 October 2026 : the road deaths of 2023 and 2024 (`tran_sf_roadus`) and the population on 1 January 2024 (`demo_pjan`).

Step 1 : Raw files

Read the death file without options. Look at its shape, its first rows and the distinct values of `unit` , `age` and `geo` . Which rows must you leave out to count each death once? Look at the population file in the same way : which age labels does it use?

Step 2 : Constants

The notebook defines `AGE_CODES` , which maps each Eurostat age code to a label, `LABELS` and `BINS` , as on the slide. Print `LABELS` . `BINS` has seven edges and makes six population bands : which label has no band?

Step 3 : Typed load

Read both files with `usecols` and with `Int64` for `TIME_PERIOD` and `OBS_VALUE` . Rename `TIME_PERIOD` , `OBS_VALUE` and `OBS_FLAG` to `year` , `value` and `flag` . Print the row counts, the number of empty values and the

population flags.

Step 4 : Death counts

Keep the rows with `unit == 'NR'`, `pers_cat == 'TOTAL'`, year 2024 and `sex == 'T'`, without `EU27_2020`. Store the sorted list of countries as `COUNTRIES`, each country's `TOTAL` row as `totals` and the remaining age rows as `counts`. Check : 33 countries, 219 age rows and 20,875 deaths.

Step 5 : Age labels

Map the age codes to labels with `AGE_CODES` and keep the columns `geo`, `age_group` and `deaths`. Check that no label is missing.

Step 6 : Population bands

Keep the population for `sex == 'T'` without `EU27_2020`. Leave out `TOTAL` and `UNK`, turn `Y_LT1` into 0 and `Y_OPEN` into 100, and band the ages with `pd.cut(..., right=False)`. Sum the result to one row per country and band, in a column `population`.

Step 7 : Band check

Check that the bands of each country add up to its published `TOTAL` population. Check one cell by hand : the under-15 population of the Netherlands is 2,714,040.

Step 8 : Grid

Build the grid of 33 countries and 7 labels with `pd.MultiIndex.from_product()`. Left-merge the bands and then the counts onto it, with `validate='one_to_one'` and `indicator=True`. Check that the grid has 231 cells and that the deaths of each country add up to its `totals`.

Step 9 : Missing counts

List the cells without a death count. Which countries are they? Why must these cells stay missing?

Step 10 : Rates

Build the summary per age group with the columns `countries`, `reported`, `missing`, `deaths`, `population` and `per_million`. `population` counts only the countries that report the group. Which age group has the highest rate, and which has the most deaths?

Step 11 : Comparisons

Compare your country rates with Eurostat's own rates, `unit == 'P_MHAB'`: how many of the 186 agree after rounding? Compare your deaths with `datasets/road-deaths-expected-summary.csv`, which was computed separately with SQL.

Step 12 : Chart

Draw one bar per age group with the rate per million. Label each bar with its deaths and population, and name what the chart leaves out in the axis label. Save the figure as `road-deaths-by-age.png` before you show it.

Step 13 : Audit

The notebook contains an analysis in the style of an AI assistant, written by the course team. It runs, and it has three defects :

1. Rows that are not country counts, the rates and the EU aggregate, multiply the joined cells
2. `errors='coerce'` turns the labels for babies and for people aged 100 and over into NaN, which removes them from the population
3. The inner join drops the deaths of unknown age and two countries, so the deaths no longer add up

Write one check per defect that fails on the proposal, without looking back at the slides. Which of your own steps prevents each defect?

If time permits :

1. Collect Steps 4–10 into a function `rates_for(sex)` ; the notebook gives its outline. Compute the rates for `'F'` and `'M'` and put the two `per_million` columns side by side. In which age group is the difference between women and men largest?
2. Write 6–10 sentences in the answer cell : counts and denominators, missing values and zeros, unknown ages, the defects of the proposal, what a rate per inhabitant cannot show, such as exposure and causes, and how another analyst reruns your notebook. If all 183 deaths of unknown age were in the 65+ group, would its rate overtake that of the 18–24 group?

Exercise 13 : Data Formats and Tools

Open `exercises/13-data-formats-and-tools/exercise.ipynb` in JupyterLab. The solution is in `solutions/13-data-formats-and-tools/solution.ipynb`.

Count the 2024 road deaths per road-user category, driver (`DRIV`), passenger (`PAS`), pedestrian (`PED`) and unknown (`UNK`), and the number of countries that report each category, with pandas, DuckDB and Polars. Use the same selection in every tool and sort the rows by category code.

Read `datasets/eurostat-road-safety-README.md` first. `eurostat-road-deaths-2023-2024.csv` is an unchanged Eurostat extract of 12,254 rows, retrieved on 10 October 2026. Each row holds one value for a combination of unit, sex, age, road-user category, country and year. Missing data are absent rows ; no value is empty.

Step 1 : Typed load

Read the file with `pd.read_csv()`. Read only these columns with `usecols` :

- `LAST UPDATE`, parsed with `parse_dates` and `date_format='%d/%m/%y %H:%M:%S'`
- The codes `sex`, `age`, `unit`, `pers_cat` and `geo`, with the dtype `category`
- `TIME_PERIOD` and `OBS_VALUE`, with the nullable dtype `Int64`
- `OBS_FLAG`, with the dtype `str`

Rename `LAST UPDATE`, `TIME_PERIOD`, `OBS_VALUE` and `OBS_FLAG` to `last_update`, `year`, `value` and `flag`. Print the dtypes and the first release time : 25 August 2026, 23:00. Check that no value is missing and that no combination of the five codes and the year occurs twice.

Step 2 : pandas

Select the rows with :

- `unit == 'NR'`, the counts
- `sex == 'T'` and `age == 'TOTAL'`
- The year 2024
- Countries only, without `EU27_2020`

Leave out the road-user category `TOTAL`. Group by `pers_cat` with `observed=True`, sum the values as `deaths` and count the rows as `countries`.

Check : 126 rows ; `DRIV` 14,078 deaths in 31 countries, `PAS` 3,007 in 31, `PED` 3,706 in 31 and `UNK` 84 in 33. Check that the categories add up to the 20,875 deaths of the `TOTAL` rows in the same selection. Why does `UNK` have 33 countries and the other categories 31?

Step 3 : Parquet

Write the loaded DataFrame to `deaths.parquet` with `index=False` and read it back. Compare the dtypes before and after. Print the stored schema with `pyarrow.parquet.read_schema()` : how does Parquet store the categories? Read only the columns `pers_cat`, `geo`, `year` and `value`.

Step 4 : DuckDB

Repeat the count with DuckDB : `con = duckdb.connect()` and a query `FROM read_parquet('deaths.parquet')` with the same filters, `GROUP BY pers_cat` and `ORDER BY pers_cat`. Name the sum `deaths` and the count `countries`, and fetch the result with `.df()`. Are the deaths and countries the same as in pandas?

Step 5 : Polars

Repeat it with a lazy query : `pl.scan_parquet('deaths.parquet')` , expressions in `.filter()` , `.group_by('pers_cat')` , `.agg()` with `pl.col('value').sum()` and `pl.len()` , and `.sort('pers_cat')` . Print `.explain()` and find the Parquet scan, the selected columns and the filter. Run the query with `.collect()` and compare the result with pandas.

Step 6 : Reflection

Write 5–8 sentences in the answer cell. Explain the selection, why the `TOTAL` rows and `EU27_2020` are left out, and why an absent row is not a zero. Which tool would you choose for interactive inspection, and which for repeated queries over a large Parquet file? Mention column selection and memory, and the difference between Arrow and Parquet.

If time permits :

- Run `demos/13-data-formats-and-tools/demo08-numba-optional.py` after `uv sync --frozen --all-groups` . Check that the three results agree before you compare the timings